

HOSPITALOS

AWS Deployment Guide

Production Infrastructure on Amazon Web Services

Architecture	Region	Est. Cost	High Availability
ECS Fargate + RDS + S3	ap-south-1 (Mumbai)	~\$143/month	Multi-AZ, Auto-scaling 2-4

Prepared for: AvnHims Hospital Implementation

Version 1.0 | May 2026 | Confidential

Table of Contents

1. Architecture Overview
2. Prerequisites
3. Cost Breakdown
4. Step 1 — AWS Account & CLI Setup
5. Step 2 — SSL Certificate (ACM)
6. Step 3 — Deploy Infrastructure (CloudFormation)
7. Step 4 — Configure Secrets
8. Step 5 — Build & Push Docker Image
9. Step 6 — Run Database Migrations
10. Step 7 — Seed Master Data
11. Step 8 — Domain & DNS Setup
12. Step 9 — CI/CD Pipeline (GitHub Actions)
13. Step 10 — Verify Deployment
14. Post-Deployment Operations
15. Monitoring & Alerting
16. Backup & Disaster Recovery
17. Scaling Guide
18. Security Checklist
19. Rollback Procedures
20. Troubleshooting
21. Environment Variables Reference

1. Architecture Overview

HospitalOS is deployed on AWS using a containerized, auto-scaling architecture designed for healthcare-grade reliability and security. The system runs in the Mumbai (ap-south-1) region for low latency to Indian hospitals.

Traffic Flow

Internet → ALB (HTTPS/443) → ECS Fargate Containers → RDS PostgreSQL

Components

Component	AWS Service	Purpose	Config
Load Balancer	Application LB	HTTPS termination, routing	Public subnets, SSL cert
Compute	ECS Fargate	Runs Next.js containers	0.5 vCPU, 1 GB RAM each
Database	RDS PostgreSQL 16	Primary data store	db.t3.small, Multi-AZ, encrypted
File Storage	S3 (Documents)	Patient docs, reports, images	Versioned, AES-256, lifecycle
Backup Storage	S3 (Backups)	DB dumps	90-day retention
Secrets	Secrets Manager	API keys, DB password, JWT	Auto-rotation capable
Logs	CloudWatch	Application & container logs	30-day retention
Container Registry	ECR	Docker image storage	Scan on push, keep last 10
Networking	VPC	Network isolation	Public + Private subnets
NAT	NAT Gateway	Outbound internet for containers	Single AZ (cost optimization)

Network Layout

Subnet	CIDR	Type	Contains
Public A	10.0.1.0/24	Public	ALB, NAT Gateway
Public B	10.0.2.0/24	Public	ALB (multi-AZ)
Private A	10.0.10.0/24	Private	ECS Tasks, RDS Primary
Private B	10.0.11.0/24	Private	ECS Tasks, RDS Standby

Security Groups

Security Group	Inbound	Source	Purpose
ALB SG	443 (HTTPS), 80 (HTTP)	0.0.0.0/0 (Internet)	Public traffic entry
App SG	3000 (Node.js)	ALB SG only	Container access
DB SG	5432 (PostgreSQL)	App SG only	Database access

NOTE: RDS is never exposed to the internet. Only ECS containers in the same VPC can reach it.

2. Prerequisites

Before starting, ensure the following are ready:

AWS Account: With admin (IAM AdministratorAccess) permissions. Create a dedicated IAM user for deployment — never use root.

AWS CLI v2: Installed and configured with your credentials. Run: `aws configure`

Docker Desktop: Installed locally for building container images. Verify: `docker --version`

Node.js 20+: For local development and testing. Verify: `node --version`

Git: Repository access to the HospitalOS codebase on GitHub.

Domain Name: A registered domain (e.g., `hims.yourhospital.com`) with DNS access to add CNAME records.

Razorpay Live Keys: Live API key and secret from the Razorpay dashboard (not test keys).

SMTP Credentials: Gmail App Password or transactional email service (Resend, Postmark, SES).

AiSensy API Key: For WhatsApp notifications (appointment reminders, OTPs).

OpenAI API Key: For AI-powered health assessment features (optional).

Install AWS CLI

```
# macOS

brew install awscli


# Linux

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

unzip awscliv2.zip && sudo ./aws/install


# Configure credentials

aws configure

AWS Access Key ID: <your-key>

AWS Secret Access Key: <your-secret>

Default region: ap-south-1

Output format: json
```

3. Cost Breakdown

Estimated monthly costs for production deployment in ap-south-1 (Mumbai):

Service	Configuration	Monthly Cost	Notes
ECS Fargate	2 tasks × 0.5 vCPU, 1 GB	~\$30	Scales to 4 under load
RDS PostgreSQL	db.t3.small, Multi-AZ	~\$50	50 GB gp3, encrypted
Application LB	1 ALB + rules	~\$20	Includes data processing
NAT Gateway	1 gateway	~\$35	Outbound traffic for containers
S3 Storage	Documents + Backups	~\$2	50 GB estimated, versioned
CloudWatch	Logs + metrics	~\$5	30-day retention
Secrets Manager	1 secret	~\$1	Per-secret pricing
TOTAL		~\$143/mo	First year: ~\$1,716

NOTE: Cost optimization: For staging environments, use db.t3.micro (\$25/mo), single container (\$15/mo), and skip NAT Gateway by placing containers in public subnets. Staging cost: ~\$65/mo.

WARNING: NAT Gateway is the second-largest cost (\$35/mo). If budget is tight, you can use VPC endpoints for S3/ECR and a cheaper NAT instance instead. This requires more setup.

STEP 1 AWS Account & CLI Setup

Create a dedicated IAM user for deployment (never use root credentials).

A. Create an IAM user for deployment

```
aws iam create-user --user-name hospitalos-deployer
```

B. Attach the required policies

```
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/AmazonECS_FullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryFullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/AmazonRDSFullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/SecretsManagerReadWrite  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/AWSCloudFormationFullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/ElasticLoadBalancingFullAccess  
aws iam attach-user-policy --user-name hospitalos-deployer \  
--policy-arn arn:aws:iam::aws:policy/IAMFullAccess
```

C. Generate access keys

```
aws iam create-access-key --user-name hospitalos-deployer
```

NOTE: Save the AccessKeyId and SecretAccessKey securely. You will need them for GitHub Actions and local CLI access.

D. Set the default region

```
aws configure set region ap-south-1  
aws configure set output json
```

STEP 2 SSL Certificate (ACM)

AWS Certificate Manager provides free SSL certificates. You must validate domain ownership via DNS.

A. Request the certificate

```
aws acm request-certificate \  
--domain-name hims.yourhospital.com \  
--validation-method DNS \  
--region ap-south-1
```

Save the CertificateArn from the output (e.g., arn:aws:acm:ap-south-1:123456789:certificate/abc-123).

B. Get the DNS validation record

```
aws acm describe-certificate \  
--certificate-arn <YOUR_CERT_ARN> \  
--query "Certificate.DomainValidationOptions[0].ResourceRecord"
```

C. Add the CNAME record to your DNS provider

Go to your domain registrar (GoDaddy, Namecheap, Cloudflare, etc.) and add the CNAME record from the output above. This proves you own the domain.

D. Wait for validation (usually 5-30 minutes)

```
aws acm describe-certificate \  
--certificate-arn <YOUR_CERT_ARN> \  
--query "Certificate.Status"
```

Wait until this returns "ISSUED".

WARNING: Do not proceed to the next step until the certificate status is ISSUED. CloudFormation will fail if the certificate is still pending.

STEP 3 Deploy Infrastructure (CloudFormation)

The CloudFormation template (aws/cloudformation.yml) creates the entire infrastructure stack in one command: VPC, subnets, security groups, ALB, ECS cluster, RDS, S3 buckets, ECR repository, IAM roles, Secrets Manager, and CloudWatch.

A. Generate a strong database password

```
DB_PASS=$(openssl rand -base64 24)
echo "SAVE THIS PASSWORD: $DB_PASS"
```

WARNING: Save this password in a secure location (password manager, not a text file). You will need it in Step 4.

B. Deploy the stack

```
aws cloudformation create-stack \
  --stack-name hospitalos-production \
  --template-body file://aws/cloudformation.yml \
  --capabilities CAPABILITY_IAM \
  --region ap-south-1 \
  --parameters \
    ParameterKey=Environment,ParameterValue=production \
    ParameterKey=AppDomain,ParameterValue=hims.yourhospital.com \
    ParameterKey=DBPassword,ParameterValue="$DB_PASS" \
    ParameterKey=CertificateArn,ParameterValue=<your-cert-arn>
```

C. Wait for completion (15-20 minutes)

```
aws cloudformation wait stack-create-complete \
  --stack-name hospitalos-production
```

D. View the outputs

```
aws cloudformation describe-stacks \
  --stack-name hospitalos-production \
  --query "Stacks[0].Outputs" \
  --output table
```

You will see these outputs:

Output Key	Description	Used In
ALBDNSName	Load balancer DNS for CNAME record	Step 8 (DNS)
DatabaseEndpoint	RDS hostname for DATABASE_URL	Step 4 (Secrets)
DocumentsBucketName	S3 bucket for patient files	App config
BackupsBucketName	S3 bucket for DB backups	Backup scripts

ECRRepositoryUri	Docker registry URL	Step 5 (Docker push)
SecretsArn	Secrets Manager ARN	Step 4 (Secrets)

STEP 4 Configure Secrets

Store all sensitive configuration in AWS Secrets Manager. The ECS task definition pulls these at runtime — no `.env` files on servers.

A. Get the RDS endpoint

```
RDS_ENDPOINT=$(aws cloudformation describe-stacks \
--stack-name hospitalos-production \
--query "Stacks[0].Outputs[?OutputKey=='DatabaseEndpoint'].OutputValue" \
--output text)
```

B. Build the DATABASE_URL

```
DATABASE_URL="postgresql://hospitalos_admin:${DB_PASS}@${RDS_ENDPOINT}:5432/hospitalos"
```

C. Update all secrets

```
aws secretsmanager update-secret \
--secret-id hospitalos/production/app \
--secret-string '{
  "DATABASE_URL": "${DATABASE_URL}",
  "DIRECT_URL": "${DATABASE_URL}",
  "JWT_SECRET": "$(openssl rand -base64 64)",
  "CONFIG_ENCRYPTION_KEY": "$(openssl rand -base64 64)",
  "RAZORPAY_KEY_ID": "rzp_live_XXXX",
  "RAZORPAY_KEY_SECRET": "your-razorpay-live-secret",
  "NEXT_PUBLIC_RAZORPAY_KEY_ID": "rzp_live_XXXX",
  "OPENAI_API_KEY": "sk-...",
  "SMTP_HOST": "smtp.gmail.com",
  "SMTP_PORT": "587",
  "SMTP_USER": "noreply@yourhospital.com",
  "SMTP_PASS": "your-app-password",
  "AISENSY_API_KEY": "your-aisensy-key",
  "CRON_SECRET": "$(openssl rand -base64 32)",
  "INTERNAL_VERIFY_TOKEN": "$(openssl rand -base64 32)"
}'
```

WARNING: Use Razorpay LIVE keys, not test keys. Test keys will not process real payments.

NOTE: For Gmail SMTP, you need an App Password (not your regular password). Go to Google Account > Security > App Passwords > Generate.

STEP 5 Build & Push Docker Image

The Dockerfile uses a multi-stage build to create a minimal production image (~200 MB). It runs as a non-root user for security.

A. Get the ECR repository URI

```
ECR_URI=$(aws cloudformation describe-stacks \
--stack-name hospitalos-production \
--query "Stacks[0].Outputs[?OutputKey=='ECRRepositoryUri'].OutputValue" \
--output text)
```

B. Login to ECR

```
aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin $ECR_URI
```

C. Build the Docker image

```
docker build -t hospitalos .
```

NOTE: First build takes 3-5 minutes. Subsequent builds use cache and take ~1 minute.

D. Tag and push to ECR

```
docker tag hospitalos:latest $ECR_URI:latest
docker tag hospitalos:latest $ECR_URI:$(git rev-parse --short HEAD)
docker push $ECR_URI:latest
docker push $ECR_URI:$(git rev-parse --short HEAD)
```

E. Force ECS to pull the new image

```
aws ecs update-service \
--cluster hospitalos-production \
--service hospitalos-production \
--force-new-deployment
```

ECS will perform a rolling deployment: start new containers with the updated image, wait for health checks to pass, then drain the old containers.

STEP 6 Run Database Migrations

Run Prisma migrations against the production database using a one-time ECS task. This creates all 120+ tables in the schema.

A. Get subnet and security group IDs

```
# Get private subnet IDs
```

```
aws ec2 describe-subnets \

--filters "Name=tag:Name,Values=hospitalos-private-*" \

--query "Subnets[].SubnetId" --output text


# Get app security group ID

aws ec2 describe-security-groups \

--filters "Name=group-name,Values=*ECS Tasks*" \

--query "SecurityGroups[0].GroupId" --output text
```

B. Run the migration task

```
aws ecs run-task \

--cluster hospitalos-production \

--task-definition hospitalos-production \

--launch-type FARGATE \

--network-configuration '{

  "awsvpcConfiguration": {

    "subnets": ["<subnet-1>", "<subnet-2>"],

    "securityGroups": ["<app-sg-id>"]

  }

}' \

--overrides '{

  "containerOverrides": [{

    "name": "hospitalos",

    "command": ["npx", "prisma", "migrate", "deploy"]

  }]

}'
```

C. Monitor the migration

```
# Get the task ARN from the run-task output, then:

aws ecs describe-tasks \

--cluster hospitalos-production \

--tasks <task-arn> \

--query "tasks[0].lastStatus"
```

NOTE: Migration takes about 1-2 minutes for the full 120+ table schema. Check CloudWatch logs if it fails:
`/ecs/hospitalos-production`

STEP 7 Seed Master Data

After migrations, seed the database with initial master data: superadmin account, default organization, departments, and chart of accounts.

A. Run the seed task

```
aws ecs run-task \  
  --cluster hospitalos-production \  
  --task-definition hospitalos-production \  
  --launch-type FARGATE \  
  --network-configuration '{...same as Step 6...}' \  
  --overrides '{  
    "containerOverrides": [{  
      "name": "hospitalos",  
      "command": ["npx", "prisma", "db", "seed"],  
      "environment": [  
        {"name": "ALLOW_SEED", "value": "true"},  
        {"name": "SEED_ADMIN_PASSWORD", "value": "<strong-password>"},  
        {"name": "SEED_SUPERADMIN_PASSWORD", "value": "<strong-password>"}  
      ]  
    }]  
  }'
```

WARNING: Change SEED_ADMIN_PASSWORD and SEED_SUPERADMIN_PASSWORD to strong, unique passwords. The defaults (password123, superadmin123) are NOT safe for production.

After seeding, hospital-specific master data (departments, doctors, services, billing tariffs, etc.) should be entered through the HospitalOS admin UI or via the Implementation Data Collection document.

STEP 8 Domain & DNS Setup

A. Get the ALB DNS name

```
aws cloudformation describe-stacks \  
  --stack-name hospitalos-production \  
  --query "Stacks[0].Outputs[?OutputKey=='ALBDNSName'].OutputValue" \  
  --output text
```

B. Add a CNAME record at your DNS provider

Record Type	Host/Name	Value/Points To	TTL
CNAME	hims.yourhospital.com	<ALB-DNS-from-above>.elb.amazonaws.com	300

C. Wait for DNS propagation (5-60 minutes)

```
# Check DNS resolution
```

```
dig hims.yourhospital.com CNAME
```

```
# Or use nslookup
```

```
nslookup hims.yourhospital.com
```

NOTE: If using Cloudflare as your DNS provider, set the proxy status to 'DNS Only' (gray cloud) to avoid SSL conflicts with ALB. You can enable Cloudflare proxy later for DDoS protection.

STEP 9 CI/CD Pipeline (GitHub Actions)

The GitHub Actions workflow (.github/workflows/deploy-aws.yml) automatically deploys on every push to main: runs tests, builds Docker image, pushes to ECR, and updates the ECS service.

A. Add secrets to GitHub repository

Go to your GitHub repo → Settings → Secrets and variables → Actions → New repository secret:

Secret Name	Value	Source
AWS_ACCESS_KEY_ID	IAM deployer access key	Step 1
AWS_SECRET_ACCESS_KEY	IAM deployer secret key	Step 1

B. Test the pipeline

```
# Make a small change and push
git add .
git commit -m "chore: enable AWS deployment"
git push origin main
```

Monitor the workflow at: github.com/<your-org>/hospital-os/actions

C. Pipeline stages

Stage	Duration	What Happens
Test	~2 min	TypeScript check, ESLint, Prisma validate
Build & Push	~4 min	Docker multi-stage build, push to ECR
Deploy	~3 min	Update ECS task, rolling deployment, health check

STEP 10 Verify Deployment

A. Check ECS service health

```
aws ecs describe-services \
  --cluster hospitalos-production \
  --services hospitalos-production \
  --query "services[0].{status:status,running:runningCount,desired:desiredCount}"
```

Expected: status=ACTIVE, running=2, desired=2

B. Hit the health endpoint

```
curl -s https://hims.yourhospital.com/api/health | python3 -m json.tool
```

Expected: {"status": "ok", "db": "connected", "timestamp": "..."}

C. Check the application

Open <https://hims.yourhospital.com> in a browser. You should see the HospitalOS login page. Login with the superadmin credentials you set during seeding.

D. Verify critical flows

Test	Action	Expected Result
Login	Superadmin login	Dashboard loads
Patient Registration	Register test patient	Patient created, UHID generated
OPD Billing	Create OPD bill	Invoice generated with correct format
Lab Order	Place lab test order	Order appears in lab queue
Payment	Process Razorpay payment	Payment captured (use Rs. 1 test)
WhatsApp	Trigger appointment reminder	Patient receives WhatsApp message
Email	Reset password flow	Email delivered

14. Post-Deployment Operations

Daily Operations

Task	Frequency	How
Check health endpoint	Automated (ALB)	ALB checks /api/health every 30s
Review CloudWatch logs	Daily	Look for errors in /ecs/hospitalos-production
Verify backup completion	Daily (first week)	Check RDS automated backup status
Monitor disk usage	Weekly	RDS storage auto-scales to 200 GB

Common Maintenance Tasks

View application logs

```
aws logs tail /ecs/hospitalos-production --follow --since 1h
```

SSH into a running container (for debugging)

```
# Enable ECS Exec first (one-time):

aws ecs update-service \

  --cluster hospitalos-production \

  --service hospitalos-production \

  --enable-execute-command

# Then exec into a container:

TASK_ARN=$(aws ecs list-tasks --cluster hospitalos-production \

  --query "taskArns[0]" --output text)

aws ecs execute-command \

  --cluster hospitalos-production \

  --task $TASK_ARN \

  --container hospitalos \

  --interactive --command /bin/sh
```

Force redeploy (without code changes)

```
aws ecs update-service \

  --cluster hospitalos-production \

  --service hospitalos-production \

  --force-new-deployment
```

15. Monitoring & Alerting

Built-in Monitoring

Service	Metrics	Where
ECS	CPU, Memory, Running task count	CloudWatch > ECS metrics
RDS	CPU, connections, IOPS, storage	CloudWatch > RDS metrics + Performance Insights
ALB	Request count, latency, 5xx errors	CloudWatch > ALB metrics
Application	Container logs, health checks	CloudWatch Logs > /ecs/hospitalos-production

Recommended Additional Setup

UptimeRobot (external monitoring, free)

- Create account at uptimerobot.com
- Add HTTP(s) monitor: <https://hims.yourhospital.com/api/health>
- Check interval: 5 minutes
- Alert contacts: your phone (SMS) + email + hospital IT

Sentry (error tracking)

```
# Install Sentry in your project:
npx @sentry/wizard@latest -i nextjs

# Add SENTRY_DSN to Secrets Manager
# Add to ECS task definition environment variables
```

CloudWatch Alarms (recommended)

Alarm	Threshold	Action
ECS CPU High	> 80% for 5 min	SNS notification to your email
RDS CPU High	> 80% for 5 min	SNS notification
RDS Storage Low	< 10 GB free	SNS notification
ALB 5xx Errors	> 10 per minute	SNS notification
Healthy Host Count	< 1	SNS notification (critical)

16. Backup & Disaster Recovery

Automated Backups (built into the stack)

Backup Type	Frequency	Retention	Recovery Time
RDS Automated Snapshot	Daily at 03:00 UTC	14 days	~15 min (restore to new instance)
RDS Multi-AZ Failover	Continuous replication	Real-time	~1-2 min (automatic)
S3 Document Versioning	On every change	All versions kept	Instant (restore version)

Manual Database Backup to S3

One-time manual backup:

```
pg_dump "$DATABASE_URL" --format=custom --no-owner | \
aws s3 cp - s3://hospitalos-production-backups/db/manual-$(date +%Y%m%d).dump
```

Restore from Backup

Option A: Restore RDS from snapshot

List available snapshots:

```
aws rds describe-db-snapshots \
--db-instance-identifier hospitalos-production \
--query "DBSnapshots[0].{ID:DBSnapshotIdentifier,Created:SnapshotCreateTime}" \
--output table
```

Restore to a new instance:

```
aws rds restore-db-instance-from-db-snapshot \
--db-instance-identifier hospitalos-restored \
--db-snapshot-identifier <snapshot-id>
```

Option B: Restore from S3 dump

Download the dump:

```
aws s3 cp s3://hospitalos-production-backups/db/manual-20260519.dump backup.dump
```

Restore:

```
pg_restore --dbname="$DATABASE_URL" --no-owner --clean backup.dump
```

17. Scaling Guide

Horizontal Scaling (more containers)

```
# Manual scale to 3 containers:

aws ecs update-service \

--cluster hospitalos-production \

--service hospitalos-production \

--desired-count 3
```

Auto-scaling is already configured: scales from 2 to 4 containers when CPU exceeds 70%.

Vertical Scaling (bigger containers)

Update the CloudFormation parameters:

Load Level	CPU	Memory	Monthly Cost Delta
Low (< 100 users)	256 (0.25 vCPU)	512 MB	Base
Medium (100-500 users)	512 (0.5 vCPU)	1024 MB	+\$0 (current)
High (500-2000 users)	1024 (1 vCPU)	2048 MB	+\$30/mo
Very High (2000+)	2048 (2 vCPU)	4096 MB	+\$90/mo

Database Scaling

Instance	vCPU	RAM	Use Case	Cost
db.t3.micro	2	1 GB	Development/testing	~\$15/mo
db.t3.small	2	2 GB	Small hospital (current)	~\$25/mo
db.t3.medium	2	4 GB	Medium hospital	~\$50/mo
db.r6g.large	2	16 GB	Large hospital, heavy queries	~\$150/mo

18. Security Checklist

Healthcare data requires strict security. Verify each item before go-live:

Secrets

- ☐ All secrets in AWS Secrets Manager (never in code or env files)
- ☐ JWT_SECRET rotated from development value
- ☐ Database password is 16+ characters
- ☐ .env is in .gitignore and NOT in the Git history
- ☐ Git history scrubbed of any leaked secrets

Network

- ☐ RDS is in private subnet (not publicly accessible)
- ☐ ECS containers are in private subnet
- ☐ ALB security group only allows 80/443
- ☐ Database security group only allows traffic from App SG
- ☐ NAT Gateway handles outbound (no public IPs on containers)

Encryption

- ☐ RDS storage encryption enabled (AES-256)
- ☐ S3 bucket encryption enabled (AES-256)
- ☐ SSL/TLS on ALB (HTTPS only, HTTP redirects)
- ☐ HSTS header enabled (already in next.config.ts)

Authentication

- ☐ Default seed passwords changed
- ☐ MFA enabled for superadmin (Speakeasy TOTP)
- ☐ Session cookies marked Secure and HttpOnly
- ☐ Rate limiting on login endpoints

Data Protection

- ☐ RDS automated backups enabled (14-day retention)
- ☐ Multi-AZ enabled for RDS (automatic failover)

- ☐ S3 versioning enabled on documents bucket
- ☐ Deletion protection enabled on RDS
- ☐ S3 public access blocked on all buckets

Monitoring

- ☐ CloudWatch logs configured (30-day retention)
- ☐ Health check endpoint (/api/health) active
- ☐ External monitoring (UptimeRobot) configured
- ☐ Error tracking (Sentry) configured

19. Rollback Procedures

Application Rollback (ECS)

ECS keeps previous task definition revisions. To roll back to the previous version:

```
# List recent task definitions:

aws ecs list-task-definitions \

  --family hospitalos-production \

  --sort DESC --max-items 5


# Roll back to a specific revision:

aws ecs update-service \

  --cluster hospitalos-production \

  --service hospitalos-production \

  --task-definition hospitalos-production:<previous-revision-number>
```

Database Rollback

If a Prisma migration causes issues, restore from the most recent RDS snapshot:

```
# This creates a NEW RDS instance from the snapshot.

# Update Secrets Manager with the new endpoint, then redeploy ECS.

aws rds restore-db-instance-from-db-snapshot \

  --db-instance-identifier hospitalos-restored \

  --db-snapshot-identifier <snapshot-id>
```

WARNING: Database rollbacks may result in data loss for records created after the snapshot. Always communicate with hospital staff before performing a database rollback.

20. Troubleshooting

ECS tasks keep restarting

Check CloudWatch logs for the crash reason. Common causes: missing env vars (check Secrets Manager), database connection refused (check security groups), or out-of-memory (increase container memory).

502 Bad Gateway from ALB

The containers are starting but not healthy yet. Check the health endpoint (/api/health). If it returns 503, the database connection is failing. Verify DATABASE_URL in Secrets Manager and that the DB security group allows traffic from the App security group.

Database connection timeout

Verify the RDS instance is running: `aws rds describe-db-instances`. Check that the App Security Group is allowed in the DB Security Group inbound rules on port 5432.

Docker build fails

Ensure you have output: 'standalone' in next.config.ts. Check that all dependencies are in package.json (not globally installed). Run `npm ci` locally to verify.

Migration fails

Check CloudWatch logs for the migration task. Common issues: schema conflicts from prior db push usage. May need to run `prisma migrate resolve --applied <migration-name>` to mark existing schema as migrated.

SSL certificate not working

Verify the certificate status is ISSUED in ACM. Check that the certificate is in ap-south-1 (same region as ALB). Ensure the CNAME validation record is still in your DNS.

DNS not resolving

Wait up to 60 minutes for propagation. Use `dig` or `nslookup` to verify. If using Cloudflare, ensure proxy is set to DNS Only (gray cloud).

Razorpay payments failing

Verify you are using LIVE keys (not test). Check that the Razorpay webhook URL is set to `https://hims.yourhospital.com/api/webhooks/razorpay`.

WhatsApp messages not sending

Verify AISENSY_API_KEY in Secrets Manager. Check that the WhatsApp Business number is registered and approved. Test with AiSensy dashboard directly.

High memory usage / OOM kills

Increase ContainerMemory in CloudFormation (1024 -> 2048). Check for memory leaks in application logs. Consider adding connection pooling (PgBouncer) if too many DB connections.

21. Environment Variables Reference

Complete list of all environment variables used by HospitalOS:

Variable	Required	Description	Stored In
DATABASE_URL	Required	PostgreSQL connection string (pooled, port 6543 or direct)	Secrets Manager
DIRECT_URL	Required	Direct PostgreSQL connection (for migrations only)	Secrets Manager
JWT_SECRET	Required	Secret key for signing JWT tokens (64+ chars)	Secrets Manager
CONFIG_ENCRYPTION_KEY	Required	Encryption key for stored integration credentials	Secrets Manager
APP_BASE_URL	Required	Full URL of the application (https://hims.hospital.com)	Task Definition
NEXT_PUBLIC_APP_URL	Required	Public URL (same as APP_BASE_URL)	Task Definition
HOSPITAL_NAME	Required	Display name of the hospital	Task Definition
ORGANIZATION_ID	Required	UUID of the organization record in the database	Task Definition
NEXT_PUBLIC_SUPABASE_URL	Optional	Supabase URL (if using Supabase Realtime)	Task Definition
NEXT_PUBLIC_SUPABASE_ANON_KEY	Optional	Supabase anon key	Task Definition
SMTP_HOST	Required	SMTP server hostname	Secrets Manager
SMTP_PORT	Required	SMTP port (587 for TLS)	Secrets Manager
SMTP_USER	Required	SMTP username/email	Secrets Manager
SMTP_PASS	Required	SMTP password or app password	Secrets Manager
RAZORPAY_KEY_ID	Required	Razorpay API key ID (live)	Secrets Manager
RAZORPAY_KEY_SECRET	Required	Razorpay API secret (live)	Secrets Manager
NEXT_PUBLIC_RAZORPAY_KEY_ID	Required	Client-side Razorpay key	Task Definition
OPENAI_API_KEY	Optional	OpenAI API key for AI features	Secrets Manager
AISENSY_API_KEY	Optional	AiSensy API key for WhatsApp	Secrets Manager
COMBIRDS_BASE_URL	Optional	AiSensy base URL	Task Definition
WHATSAPP_VERIFY_TOKEN	Optional	Webhook verification token	Secrets Manager
CRON_SECRET	Required	Secret for cron job endpoints	Secrets Manager
INTERNAL_VERIFY_TOKEN	Required	Internal API verification token	Secrets Manager
AWS_S3_BUCKET	Required	S3 bucket name for documents	Task Definition
AWS_REGION	Required	AWS region (ap-south-1)	Task Definition

End of Document

HospitalOS AWS Deployment Guide v1.0

For questions or support, contact the development team.